



ESTRUTURA DE DADOS

Aula 3 – Vetores Ordenados

Prof. Evandro Luiz

VETORES ORDENADOS

- Os elementos devem ser do mesmo tipo;
- O tamanho é fixado na declaração (criação) do vetor;
- Os dados estão organizados na ordem ascendente de valores-chave;
- O menor valor no índice 0 e cada célula mantendo um valor maior que a célula abaixo;
- Vantagem: agiliza os tempos de pesquisa.



VETORES ORDENADOS - INSERÇÃO

- Pesquisar uma média de $N/2$ elementos (pesquisa linear)
 - Pior caso: N
- Mover os elementos restantes ($N/2$ passos)
 - Pior caso: N

3						
1	2	4	5			
1	2	4		5		
1	2		4	5		
1	2	3	4	5		

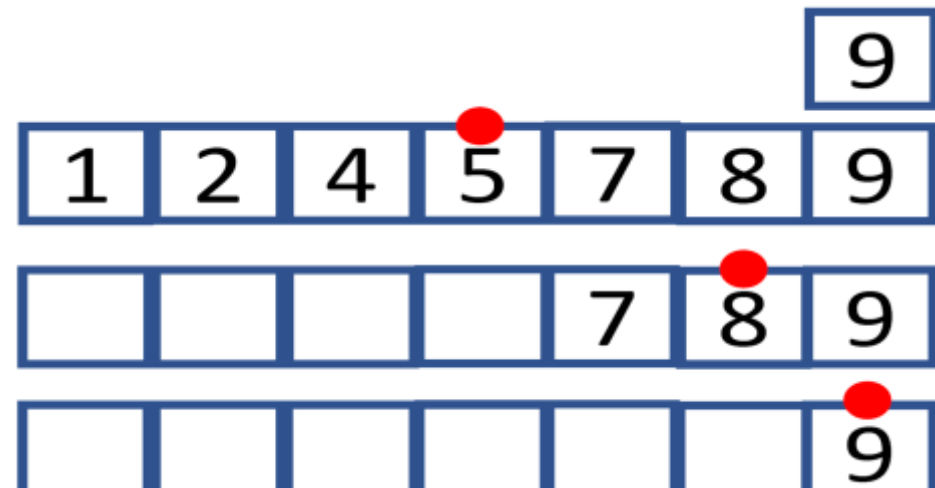
VETORES ORDENADOS - PESQUISA

- A pesquisa termina quando o primeiro item maior que o valor de pesquisa é atingido;
- Como o vetor está ordenado, o algoritmo sabe que não há mais necessidade de procurar.

1	2	4	5	8		
---	---	---	---	---	--	--

VETORES ORDENADOS - PESQUISA BINÁRIA

- Suponha um vetor ordenado contendo números de 1 até 9;
- Pesquisar o número 9;
- Obtém a metade de 1 até 9 = 5
- 5 é o número pesquisado? Não
- 9 é menor ou maior do que 5? Maior
- Obtém a metade de 7 até 9 = 8
- 8 é o número pesquisado? Não
- 9 é menor ou maior do que 8? Maior
- Neste instante o algoritmo tenta obter a metade de 9 até 9 = 9
- 9 é o número pesquisado? **Sim**



VETORES ORDENADOS - PESQUISA BINÁRIA x LINEAR

Faixa	Comparações Binária	Comparações Linear (N/2)
10	4	5
100	7	50
1.000	10	500
10.000	14	5.000
100.000	17	50.000
1.000.000	20	500.000
10.000.000	24	5.000.000
100.000.000	27	50.000.000
1.000.000.000	30	500.000.000

VETORES ORDENADOS - EXCLUSÃO

- Semelhante ao Vetor Não Ordenado;
- Primeiro executa uma pesquisa linear para encontrar o elemento que deseja excluir;
- **O algoritmo pode terminar na metade do caminho se não encontrar o item;**
- Após a exclusão, todos os elementos precisam ser deslocados para ocupar a lacuna deixada pelo elemento excluído.

VETORES ORDENADOS - DISCUSSÕES

- A principal vantagem é que os tempos de pesquisa são muito mais rápidos do que em um vetor não ordenado;
- A inserção leva mais tempo, pois os itens de dados devem ser movidos para criar espaço;
- As remoções são lentas tanto nos vetores ordenados quanto nos não ordenados, pois os itens devem ser movidos para preencher as lacunas “buracos”;
- Vetores ordenados são úteis quando pesquisas são frequentes, mas inserções e remoções não são.

OPERAÇÕES COM PSEUDO-CÓDIGO

Criação da Classe:

 <i>Vetor Ordenado</i>
Capacidade: inteiro Última Posição: inteiro = -1 Valores: inteiro[]*Capacidade
Imprimir() Inserir(valor) Pesquisa Linear(valor) Pesquisa Binária(valor) Excluir(valor)

OPERAÇÕES COM PSEUDO-CÓDIGO

Algoritmo 1: Inserção de valores

inserir(valor):

se ultima_posição = capacidade - 1 **então:**

imprime('Mensagem de Vetor Cheio')

return

//INICIA A BUSCA PELO POSIÇÃO DA INSERÇÃO

 posição <- 0

para cada i no range(ultima_posição + 1) **faça:**

 posição <- i

se valores[i] > valor **então:**

break

se i = ultima_posição **então:**

 posição <- i + 1

 x <- ultima_posição

Enquanto x >= posição **faça:**

 valores[x + 1] <- valores[x]

 x <- x - 1

 valores[posição] <- valor

 ultima_posição <- ultima_posição + 1

OPERAÇÕES COM PSEUDO-CÓDIGO

Algoritmo 1: Inserção de valores

inserir(valor):

se ultima_posição = capacidade - 1 **então:**

imprime('Mensagem de Vetor Cheio')

return

posição <- 0

para cada i no range(ultima_posição + 1) **faça:**

posição <- i

se valores[i] > valor **então:**

break

se i = ultima_posição **então:**

posição <- i + 1

x <- ultima_posição

Enquanto x >= posição **faça:**

valores[x + 1] <- valores[x]

x <- x - 1

valores[posição] <- valor

ultima_posição <- ultima_posição + 1

- O algoritmo procura a posição onde o novo valor deve ser inserido. Ele percorre o vetor até ultima_posição e compara cada elemento com o valor a ser inserido.

- Se encontrar um elemento maior que o valor, ele interrompe a busca (com o break).

- Se chegar ao final do vetor (i = ultima_posição), significa que o valor é maior do que todos os elementos existentes, então a próxima posição será i + 1.

- Deslocar elementos para abrir espaço.

- Para inserir o novo valor, todos os elementos a partir da posição até o final do vetor precisam ser deslocados uma posição para a direita para abrir espaço para o novo valor.

- Insere o valor e atualiza a última posição

OPERAÇÕES COM PSEUDO-CÓDIGO

Algoritmo 1: Inserção de valores

1 - Verifica se o vetor está cheio

se `ultima_posição = capacidade - 1` então:

`imprime('Mensagem de Vetor Cheio')`

`return`

Se a última posição disponível já foi ocupada, não é possível inserir outro elemento.

Por exemplo, se: `capacidade = 5` as posições são:

0 1 2 3 4

Então, quando:

`ultima_posição = 4`

o vetor está cheio.

OPERAÇÕES COM PSEUDO-CÓDIGO

Algoritmo 1: Inserção de valores

2 - Inicia a busca pela posição correta

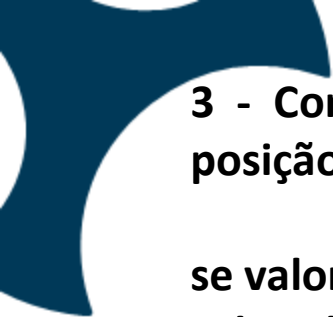
posição $\leftarrow$ 0

Inicialmente assumimos que o novo valor será colocado na posição 0.

Depois:

para cada i no range(ultima_posição + 1) faça:

Percorremos os elementos que já estão armazenados.



**3 - Compara o valor com os elementos
posição <- i**

**se valores[i] > valor então:
break**

Aqui está a parte principal.

Estamos procurando o primeiro elemento maior que o valor que queremos inserir.

Por exemplo, temos:

[10, 20, 30, 40]

e queremos inserir: 25

As comparações serão:

10 > 25? Não

20 > 25? Não

30 > 25? Sim

Então: posição = 2

O 25 deverá entrar na posição 2.

4 - Caso o valor seja maior que todos

Temos:

se $i = \text{ultima_posição}$ então:
 $\text{posição} \leftarrow i + 1$

Isso trata o caso em que o novo valor deve ser colocado no final do vetor.

Exemplo:

[10, 20, 30, 40]

Inserindo: 50

Como 50 é maior que todos:

$\text{posição} = 4$

5 - Deslocando os elementos

Agora temos:

```
x <- ultima_posição
```

e:

Enquanto $x \geq \text{posição}$ faça:

```
valores[x + 1] <- valores[x]
```

```
x <- x - 1
```

Essa parte é muito importante.

Precisamos abrir espaço para o novo elemento.

Imagine:

[10, 20, 30, 40]

Inserir: 25

A posição correta é 2.

Primeiro deslocamos 40:

[10, 20, 30, 40, 40]

Depois deslocamos 30:

[10, 20, 30, 30, 40]

Agora existe espaço na posição 2.



6 - Insere o valor
valores[posição] <- valor

Então:

valores[2] <- 25

Resultado:

[10, 20, 25, 30, 40]

7. Atualiza a última posição

ultima_posição <- ultima_posição + 1

Isso informa que agora existe mais um elemento armazenado no vetor.

OPERAÇÕES COM PSEUDO-CÓDIGO

Algoritmo 2: Impressão dos valores

imprimir():

se ultima_posicao = -1 **então:**

imprime('Mensagem de vetor vazio')

senão:

para cada i no **range**(ultima_posicao + 1) **faça:**

imprime(i, ' - ', valores[i])

OPERAÇÕES COM PSEUDO-CÓDIGO

Algoritmo 3: Pesquisa Linear

pesquisa_linear(valor):

para cada i no range(ultima_posição + 1) faça:

se valores[i] > valor então:

return -1

se valores[i] = valor então:

return i

se i = ultima_posição então:

return -1

OPERAÇÕES COM PSEUDO-CÓDIGO

Algoritmo 4: Pesquisa Binária

pesquisa_binaria(valor):

limite_inferior <- 0

limite_superior <- ultima_posição

Enquanto True **faça:**

posição_atual <- (limite_inferior + limite_superior) / 2

Se achou

se valores[posição_atual] = valor **então:**

return posição_atual

Se não achou

senão se limite_inferior > limite_superior **então:**

return -1

Divide os limites

senão:

 # Limite inferior

se valores[posição_atual] < valor **então:**

 limite_inferior <- posição_atual + 1

 # Limite superior

senão:

 limite_superior <- posição_atual - 1

O loop continua enquanto limite_inferior for menor ou igual a limite_superior.

A posição_atual é calculada como a média dos limites inferior e superior, usando a divisão inteira (//) para garantir que ela seja um índice válido.

Se o valor na posição_atual for igual ao valor procurado, ele retorna essa posição.

Se o valor na posição_atual for menor que o valor buscado, o limite inferior é atualizado para um índice acima da posição atual.

Se o valor na posição_atual for maior, o limite superior é atualizado para um índice abaixo da posição atual.

OPERAÇÕES COM PSEUDO-CÓDIGO

Algoritmo 5: Excluir um valor

excluir(valor):

posição <- **pesquisar**(valor)

se posicao = -1 **então:**

return -1

senão:

para cada i no **range**(posicao, ultima_posicao) **faça:**

valores[i] <- valores[i + 1]

ultima_posicao <- ultima_posicao - 1